Fully Autonomous Drone for Adaptive Obstacle Avoidance

22AIE448 Introduction to Data driven Control of Drones

Team members :

K.Sasank - CB.SC.U4AIE24026

Varshitha.P- CB.SC.U4AIE24144

S.Tagur Misra - CB.SC.U4AIE24150

Introduction :

The proposed system is designed to achieve safe, reliable, and real-time autonomous navigation while reducing computational complexity and power consumption.

While flying in unknown environments, drones may encounter walls, obstacles, narrow corridors, and corners, making safe navigation a major challenge.

The adaptive controller enables the drone to detect obstacles, avoid collisions, navigate through corners, and escape deadlock situations without creating a map of the environment.

Problem statement :

Autonomous drones often operate in complex and unknown environments where obstacles such as walls, corners, and narrow passages increase the risk of collisions.

Drones often face complex situations such as obstacles, narrow corridors, corners, and deadlocks, where traditional methods may struggle to navigate efficiently.

Many existing systems require multiple sensors and complex processing, increasing the cost, weight, and power consumption of autonomous drones.

Objective :

To develop an autonomous drone system capable of navigating safely in real-world environments without human intervention.

To design an adaptive obstacle avoidance system using two lightweight LiDAR sensors for real-time obstacle detection.

To implement a computationally efficient neural control algorithm that enables obstacle avoidance with low processing requirements

To develop a lightweight drone platform suitable for implementing the adaptive obstacle avoidance controller using minimal sensing hardware.

Architecture :

Two LiDAR sensors are mounted at the front of the drone with a defined orientation angle to detect obstacles and measure their distances.

The normalized data is processed by the adaptive neural obstacle avoidance controller, which uses short-term memory and synaptic plasticity to navigate safely through obstacles, corners, and deadlocks

The controller generates yaw (steering) commands, which are sent from the Raspberry Pi to the Pixhawk flight controller via the DroneKit–MAVProxy–MAVLink communication framework.

The Pixhawk flight controller stabilizes the drone and computes the motor control signals, enabling smooth autonomous navigation and collision avoidance.

Simulated LiDAR → Sensor Normalization → Adaptive Neural Controller → Yaw/Movement Command → PyBullet Drone → Environment Feedback

Dual LiDAR Sensors

Two lightweight LiDAR sensors are mounted at the front of the drone to detect obstacles.

The sensors are oriented at a specific angle to provide obstacle-distance information to the neural controller.

What does it do?

Continuously measures the distance to obstacles.
Provides real-time distance information to the processing unit.

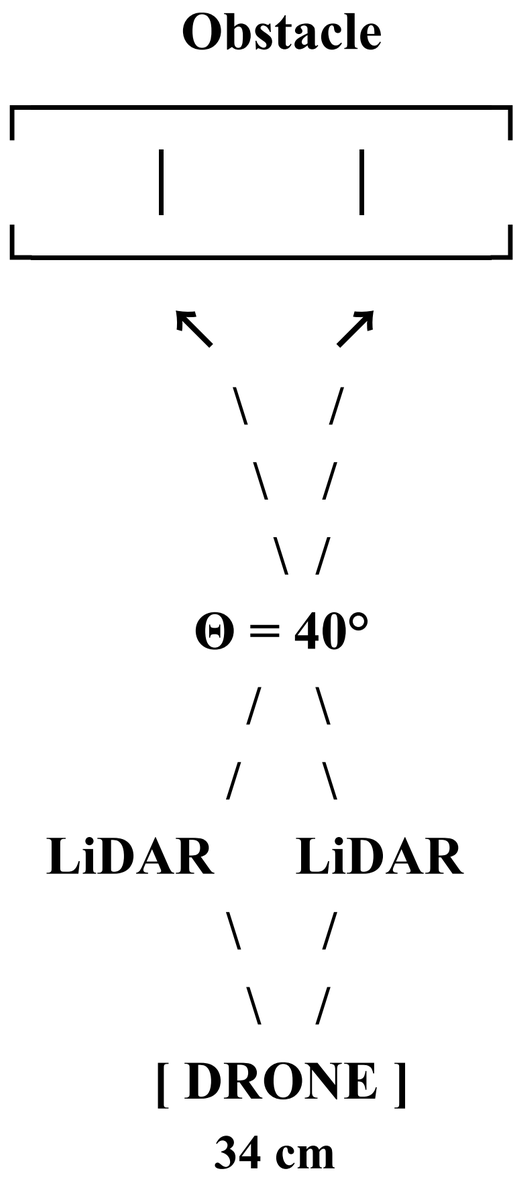
LiDAR Sensor Orientation – Mathematical Design

Sensor geometry :

$W_{obj} \geq 2\tan(\Theta/2)(L_x + L_d/2)$

Where:

- W_{obj} = minimum detectable obstacle width
- Θ = angle between the two LiDAR sensors
- L_x = distance between drone and obstacle
- L_d = drone width



Sensor Data Processing

The raw distance values cannot be directly used by the controller.

What does it do?

Raw LiDAR measurements are mapped to [-1, +1]. A value near -1 represents no obstacle within the sensing range, while +1 represents a nearby obstacle (~20 cm).

Adaptive Neural Obstacle Avoidance Controller

Instead of using complex AI models such as CNN or SLAM, the paper proposes a lightweight neural controller that requires very little computation while still making intelligent navigation decisions.

What does it do?

Receives processed LiDAR inputs.

The controller processes the LiDAR inputs and generates a yaw command that determines the steering direction of the drone.

$$S_L,\ S_R \in [-1, 1]$$

SL = left LiDAR signal

SR = right LiDAR signal

−1 = no obstacle within sensing range

+1 = obstacle is very close

Neural State Update:

$$O_1(t) = \tanh\left[b_1 O_1(t-1) + c O_2(t-1) + I_g S_L(t)\right]$$

$$O_2(t) = \tanh\left[b_2 O_2(t-1) + c O_1(t-1) + I_g S_R(t)\right]$$

O1(t),O2(t) – Current outputs of the two neurons
O1(t−1),O2(t−1) – Previous neuron states; provide short-term memory
b1,b2 – Self-feedback parameters controlling the influence of previous neuron activity
c – Mutual inhibitory coupling between the two neurons
SL,SR – Normalized left and right LiDAR signals
Ig – Input gain controlling the influence of LiDAR signals
t – Current time step; t−1 represents the previous time step
tanh – Nonlinear activation function that bounds the neuron output between −1 and +1

The equations combine current LiDAR information, previous neural states, and interaction between the two neurons to generate the current neural response. The recurrent terms provide short-term memory, while the difference between the neuron outputs is later used to generate the drone's steering/yaw command.

Steering Calculation :

$$Steering = \frac{O_1 - O_2}{2}$$

It calculates the difference between the two neuron outputs.

Why ?

The difference tells the controller which turning direction is preferred.

If one neuron becomes more active than the other, the steering value moves away from zero.

$$YawRate = -K_{turn} \times Steering$$

Left & Right LiDAR Measurements

↓

Two-Neuron Recurrent Neural Network

↓

Short-Term Memory + Mutual Inhibition

↓

Neural Output Difference

↓

Yaw Command

↓

Drone Turns to Avoid Obstacle

↓

Synaptic Plasticity → Online Adaptation

↺ Feedback to LiDAR

Short-Term Memory & Synaptic Plasticity

In corners or dead-end situations, obstacles may temporarily disappear from the LiDAR's field of view. Without memory, the drone could stop turning too early and collide with a wall.

What does it do?

short-Term Memory allows the drone to remember its previous turning action and continue turning even after the obstacle is no longer detected.

Synaptic Plasticity continuously adjusts the neural controller's behavior according to the environment, improving navigation through complex areas

Mathematical Explanation :

The controller remembers previous sensor information using short-term memory. The synaptic weights are updated continuously to adapt to changing environments.

Short term memory:

$$h_t = f(x_t + h_{t-1})$$

xt = Current sensor input

ht−1 = Previous memory state

ht = Updated memory state

Raspberry Pi Zero

A processing unit is required to execute the obstacle avoidance algorithm onboard the drone.

What does it do?

Executes the adaptive neural controller.

Processes LiDAR inputs.

Generates navigation commands.

Communicates with the flight controller

MAVLink Communication Framework

The Raspberry Pi cannot directly control the drone hardware.

What does it do?

DroneKit provides APIs to generate flight commands.

MAVProxy provides the MAVLink communication layer, while DroneKit provides a higher-level Python API for drone commands.
MAVLink transmits the commands to the Pixhawk flight controller.

Closed-Loop Control System

The adaptive obstacle avoidance system operates as a closed-loop system in which LiDAR sensory feedback is continuously used by the neural controller to generate steering commands.

Environment

↓

LiDAR measurements

↓

Neural controller

↓

Yaw command

↓

Drone movement

↓

Environment changes

↺ LiDAR feedback

ESCs and BLDC Motors

The drone requires actuators to execute the movement commands.

What does it do?

ESCs regulate the speed of each BLDC motor.

The motors generate the required thrust.

ESCs and BLDC motors enable the drone to execute obstacle avoidance manoeuvres.

Parameter Values:

Drone width Ld : 34 cm

Safety margin Lm : 6 cm

LiDAR sensing range Ls : 50 cm

LiDAR orientation angle Θ : 40

Drone speed : 0.1 m/s

Obstacle turning distance : 15 cm

Minimum obstacle width: 24 cm

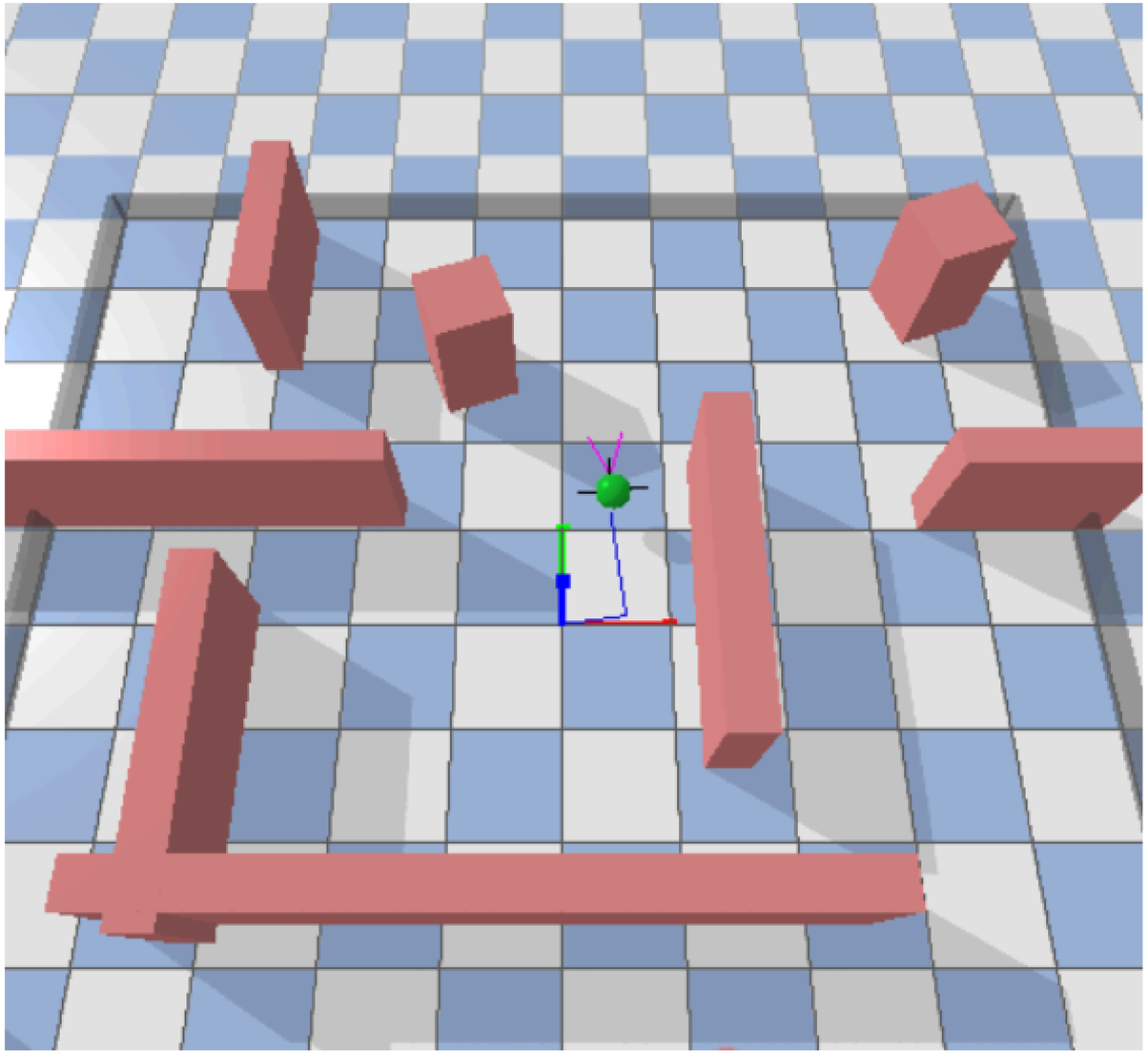
Simulation Setup

The proposed autonomous navigation system is implemented in a simulated environment to verify the obstacle avoidance capability of the controller.

Simulation Environment

- A virtual environment is created with walls and obstacles representing real-world navigation conditions.
- The drone is equipped with two simulated LiDAR sensors for obstacle-distance measurement.
- LiDAR readings are normalized and supplied to the two-neuron recurrent controller.
- The controller generates yaw commands based on the sensor inputs.
- The drone's movement are recorded to evaluate obstacle avoidance.

Autonomous Drone Simulation



Simulation Results

- The drone successfully detects obstacles using the two front LiDAR sensors.
- The neural controller converts sensor information into steering/yaw commands.
- The drone changes its direction when an obstacle is detected.
- The travelled path demonstrates the resulting obstacle-avoidance behaviour.

Base paper :

Development of Autonomous Drones for Adaptive Obstacle Avoidance in Real World Environments

Link:

https://ieeexplore.ieee.org/document/8491889

Thank You